



COL7378: Principles of Autonomous Systems

Semester II, 2025-26

Sate Estimation - II

Rohan Paul

Today's lecture

- Last Class
 - State Estimation - I
 - Recursive State Estimation
 - Bayes Filter
- This Class
 - State Estimation - II
 - Kalman Filter
 - Extended Kalman Filter
- References
 - Probabilistic Robotics Ch 3 (Sec. 3.1-3.3)
 - AIMA Ch 15 (Sec. 15.4)

Acknowledgements

These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Nicholas Roy, Wolfram Burgard, Dieter Fox, Sebastian Thrun, Siddharth Srinivasa, Dan Klein, Pieter Abbeel and others.

State Estimation: Continuous Variables

- Bayes Filter till now
 - Discrete state variables
 - E.g., door open or closed.
 - Discrete conditional probability tables.
- Continuous variables
 - Example: we receive continuous measurements of the position or height and seek an estimate. Control the vehicle via velocities.
- Kalman Filter
 - Special case of a Bayes' filter for handling continuous variables.
 - Assumes that the motion model (dynamics/control) and the sensor model is linear Gaussian.
 - E.g., estimating the belief over the location of the agent given the sequence of observations and controls.

Multivariate Gaussians

- Distribution over a vector of variables
 - E.g., the agent's state in our case.
- Mean vector
 - Expected value of each variable.
- Covariance matrix
 - Covariance between each pair of elements of a given random vector.
 - Diagonals contain variance of each variable in the state.
 - Symmetric and positive semi-definite.

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu) \right)$$

$$\mathbb{E}_X[X_i] = \int x_i p(x; \mu, \Sigma) dx = \mu_i$$

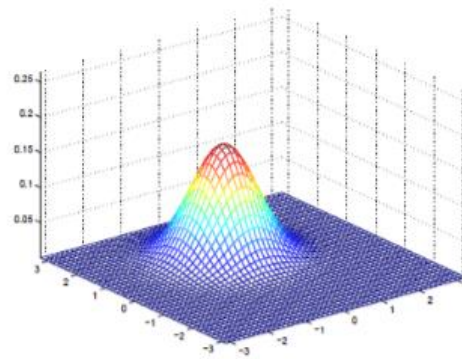
$$\mathbb{E}_X[X] = \int x p(x; \mu, \Sigma) dx = \mu$$

$$\mathbb{E}_X[(X_i - \mu_i)(X_j - \mu_j)] = \int (x_i - \mu_i)(x_j - \mu_j) p(x; \mu, \Sigma) dx = \Sigma_{ij}$$

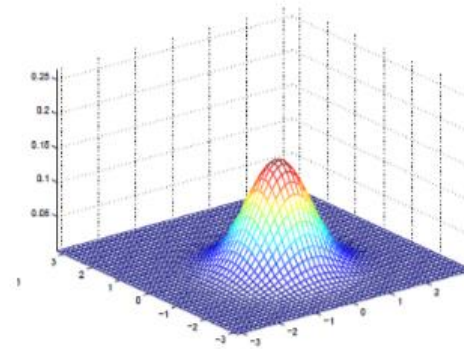
$$\mathbb{E}_X[(X - \mu)(X - \mu)^\top] = \int [(X - \mu)(X - \mu)^\top] p(x; \mu, \Sigma) dx = \Sigma$$

Multivariate Gaussians: Examples

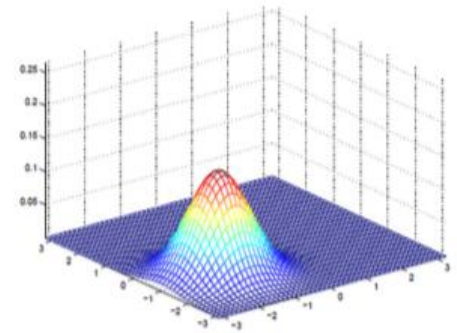
- Varying the mean or origin of the distribution.



- $\mu = [1; 0]$
- $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$



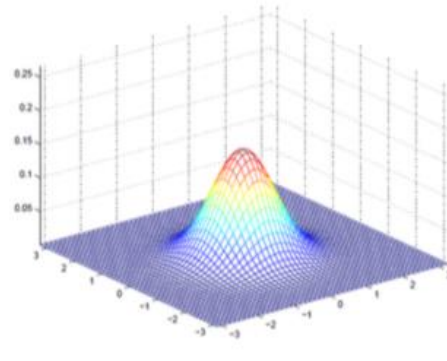
- $\mu = [-.5; 0]$
- $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$



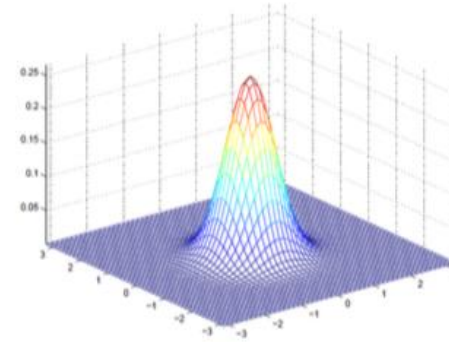
- $\mu = [-1; -1.5]$
- $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Multivariate Gaussians: Examples

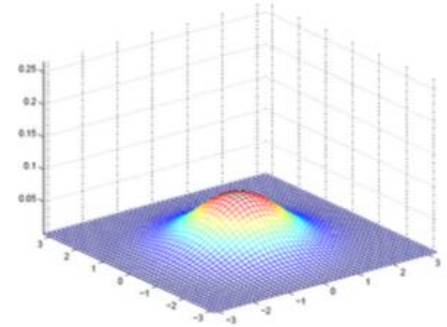
- Changing the variance in the state variables.



■ $\mu = [0; 0]$
■ $\Sigma = [1 \ 0; 0 \ 1]$



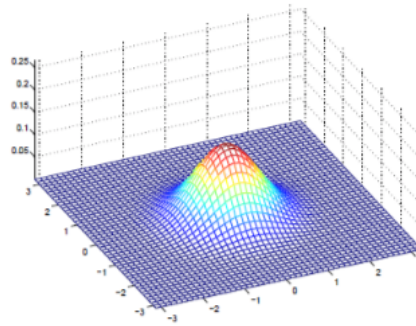
■ $\mu = [0; 0]$
■ $\Sigma = [.6 \ 0; 0 \ .6]$



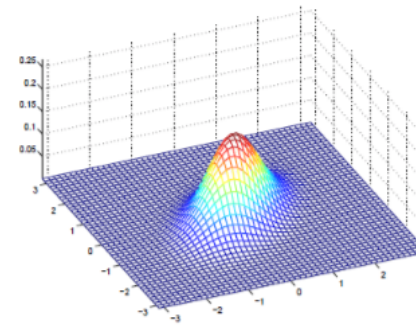
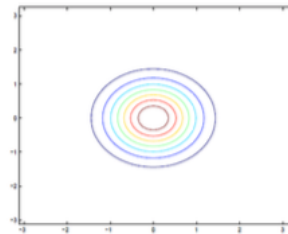
■ $\mu = [0; 0]$
■ $\Sigma = [2 \ 0; 0 \ 2]$

Multivariate Gaussians: Examples

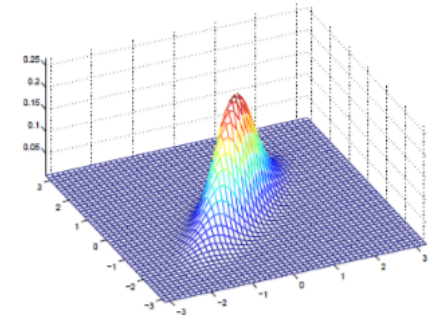
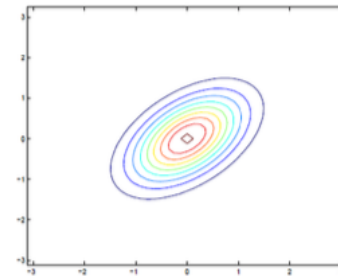
- Changing the variance in the off-diagonal elements.
 - Model variance *between* state variables.



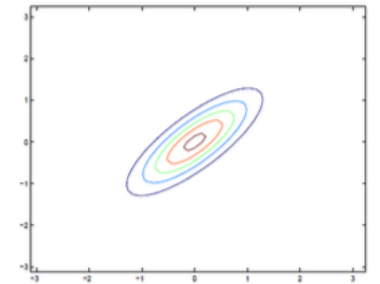
- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$



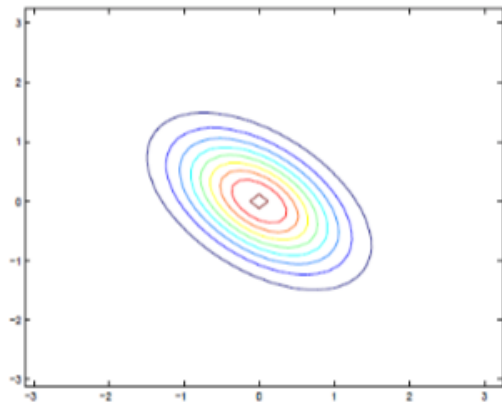
- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$



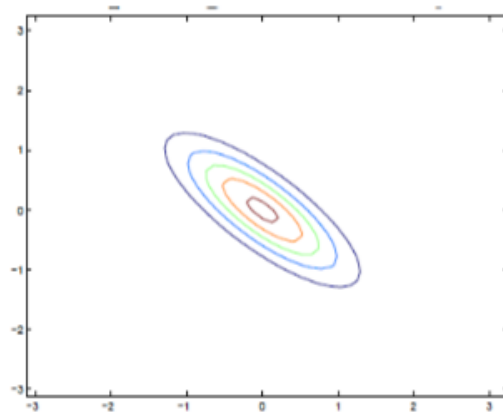
- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$



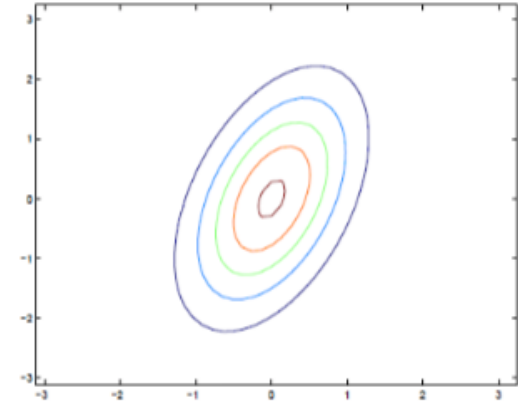
Multivariate Gaussians: Examples



- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}$



- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}$



- $\mu = [0; 0]$
- $\Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}$

Joint Gaussian PDFs: Variable Partitioning

- Partition the random vector as variables as (X, Y).
 - Notice the block structure.
- Why?
 - Later, we would need to marginalize or condition on *some* of the variables.

$$\mathcal{N}(\mu, \Sigma) = \mathcal{N}\left(\begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}, \begin{bmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{bmatrix}\right)$$

$$p\left(\begin{bmatrix} x \\ y \end{bmatrix}; \mu, \Sigma\right) = \frac{1}{(2\pi)^{(n/2)}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2} \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}\right)^\top \begin{bmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{bmatrix}^{-1} \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}\right)\right)$$

$$\begin{aligned}\mu_X &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[X] \\ \mu_Y &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[Y] \\ \Sigma_{XX} &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[(X - \mu_X)(X - \mu_X)^\top] \\ \Sigma_{YY} &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[(Y - \mu_Y)(Y - \mu_Y)^\top] \\ \Sigma_{XY} &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[(X - \mu_X)(Y - \mu_Y)^\top] = \Sigma_{YX}^\top \\ \Sigma_{YX} &= \mathbb{E}_{(X,Y) \sim \mathcal{N}(\mu, \Sigma)}[(Y - \mu_Y)(X - \mu_X)^\top] = \Sigma_{XY}^\top\end{aligned}$$

Joint Gaussian PDFs: Marginalization

- Marginalization
 - Integrating out the effect of a (sub)-set of variables.
 - Resulting is a normal distribution over a smaller set of variables.
 - The resulting distribution is still Gaussian.

If

$$(X, Y) \sim \mathcal{N}(\mu, \Sigma) = \mathcal{N}\left(\begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}, \begin{bmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{bmatrix}\right)$$

Then

$$\begin{aligned} X &\sim \mathcal{N}(\mu_X, \Sigma_{XX}) \\ Y &\sim \mathcal{N}(\mu_Y, \Sigma_{YY}) \end{aligned}$$

Joint Gaussian PDFs: Conditioning

- Conditioning

- Certain variables are observed (known and instantiated with observed values).
- We seek the distribution over the remaining set of variables.
- Conditioning a Gaussian results in another Gaussian distribution.

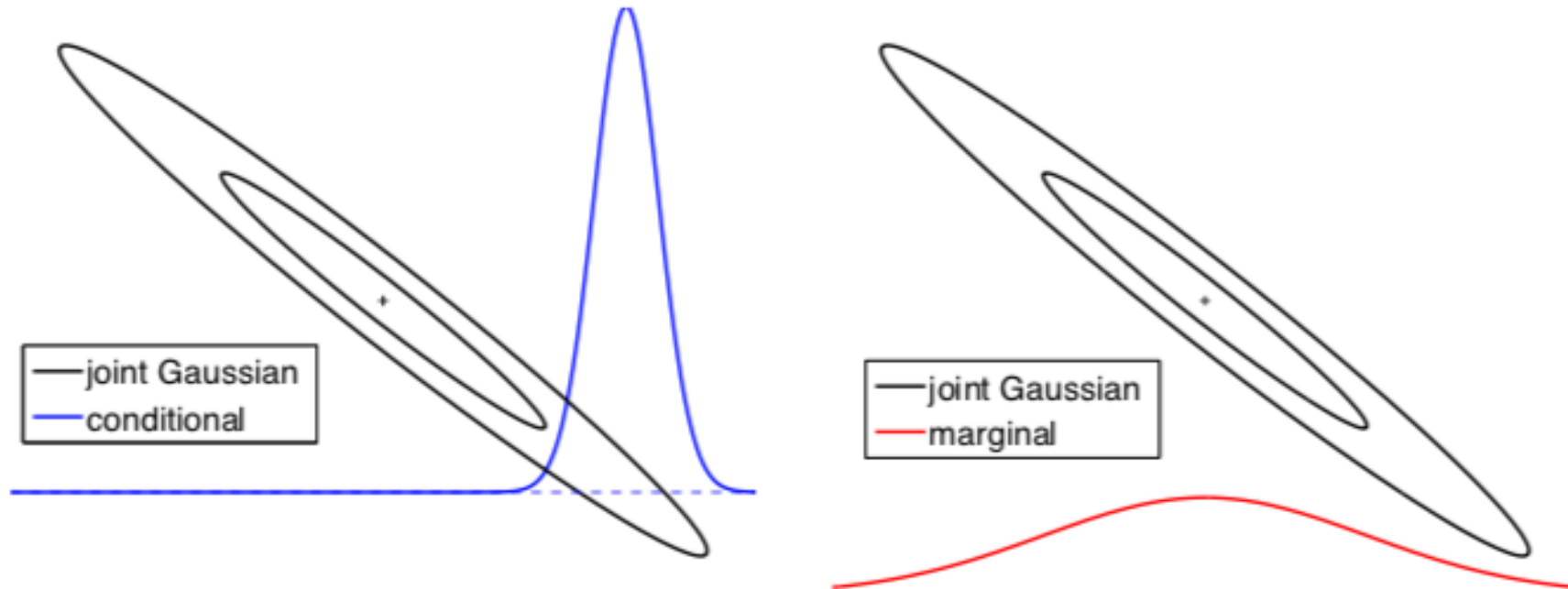
If

$$(X, Y) \sim \mathcal{N}(\mu, \Sigma) = \mathcal{N}\left(\begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}, \begin{bmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{bmatrix}\right)$$

Then

$$\begin{aligned} X|Y = y_0 &\sim \mathcal{N}(\mu_X + \Sigma_{XY}\Sigma_{YY}^{-1}(y_0 - \mu_Y), \Sigma_{XX} - \Sigma_{XY}\Sigma_{YY}^{-1}\Sigma_{YX}) \\ Y|X = x_0 &\sim \mathcal{N}(\mu_Y + \Sigma_{YX}\Sigma_{XX}^{-1}(x_0 - \mu_X), \Sigma_{YY} - \Sigma_{YX}\Sigma_{XX}^{-1}\Sigma_{XY}) \end{aligned}$$

Conditionals and Marginals of a Gaussian Distribution



Both the **conditionals** and the **marginals** of a joint Gaussian are again Gaussian.

Other Properties

- Linear transformation
- Product

$$\left. \begin{array}{l} X \sim N(\mu, \sigma^2) \\ Y = aX + b \end{array} \right\} \Rightarrow Y \sim N(a\mu + b, a^2\sigma^2)$$

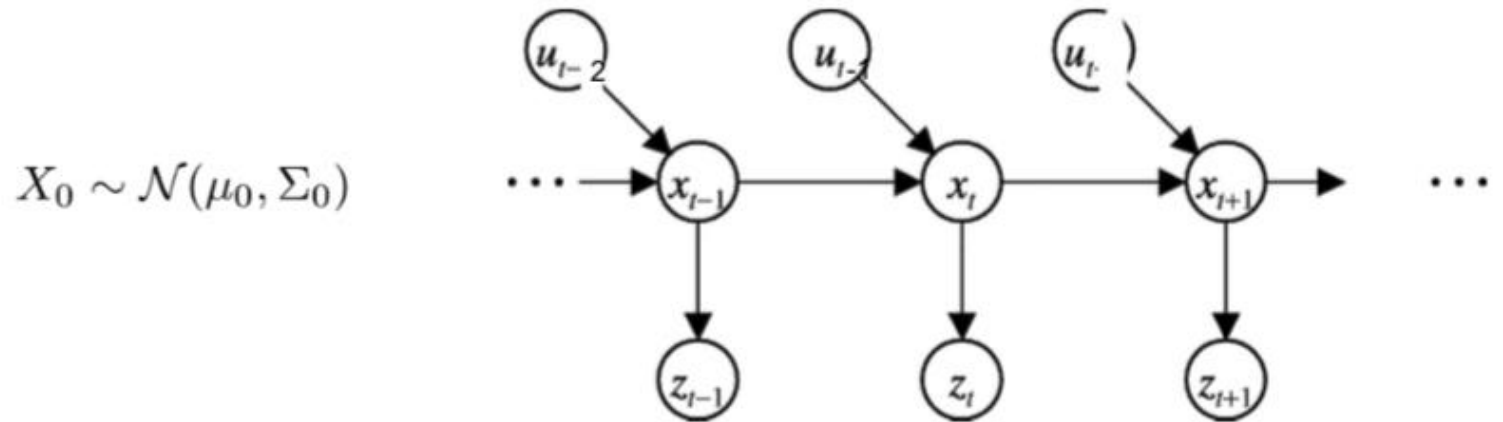
$$\left. \begin{array}{l} X_1 \sim N(\mu_1, \sigma_1^2) \\ X_2 \sim N(\mu_2, \sigma_2^2) \end{array} \right\} \Rightarrow p(X_1) \times p(X_2) \sim N\left(\frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} \mu_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} \mu_2, \frac{1}{\sigma_1^{-2} + \sigma_2^{-2}}\right)$$

$$\left. \begin{array}{l} X \sim N(\mu, \Sigma) \\ Y = AX + B \end{array} \right\} \Rightarrow Y \sim N(A\mu + B, A\Sigma A^T)$$

$$\left. \begin{array}{l} X_1 \sim N(\mu_1, \Sigma_1) \\ X_2 \sim N(\mu_2, \Sigma_2) \end{array} \right\} \Rightarrow p(X_1) \times p(X_2) \sim N\left(\frac{\Sigma_2}{\Sigma_1 + \Sigma_2} \mu_1 + \frac{\Sigma_1}{\Sigma_1 + \Sigma_2} \mu_2, \frac{1}{\Sigma_1^{-1} + \Sigma_2^{-1}}\right)$$

Kalman Filter

- Provided
 - A belief over the initial state
 - Sensor model is linear Gaussian
 - Motion model is linear Gaussian
- What is our goal
 - Estimate a belief over the latent state at time t.

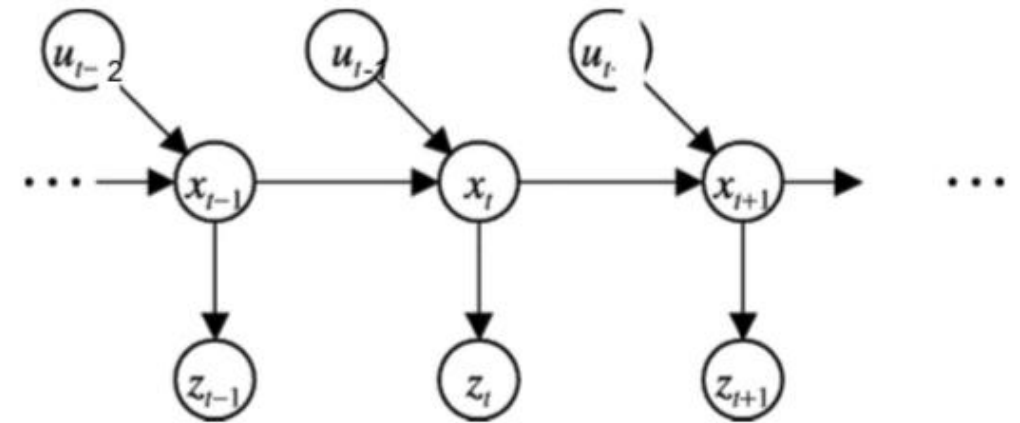


$$\begin{aligned} X_{t+1} &= A_t X_t + B_t u_t + \varepsilon_t & \varepsilon_t &\sim \mathcal{N}(0, Q_t) \\ Z_t &= C_t X_t + d_t + \delta_t & \delta_t &\sim \mathcal{N}(0, R_t) \end{aligned}$$

Kalman Filter: Components

- A_t Matrix
 - Size $(n \times n)$ that describes how the state evolves from 1 to t without controls or noise.
- B_t Matrix
 - Size $(n \times l)$ that describes how the control u changes the state from $t-1$ to t .
- Epsilon
 - Random variable (size n) representing the process noise that is assumed to be independent and normally distributed with covariance Q_t (size $n \times n$).
- C_t Matrix
 - Size $(k \times n)$ that describes how to map the state x_t to an observation z_t .
- d_t Vector
 - Size (k) constant offset added. Often explicit mention of d is dropped from the sensor model.
- Delta
 - Random variable (size k) representing the measurement noise that is assumed to be independent and normally distributed with covariance R_t (size $k \times k$).

$$X_0 \sim \mathcal{N}(\mu_0, \Sigma_0)$$

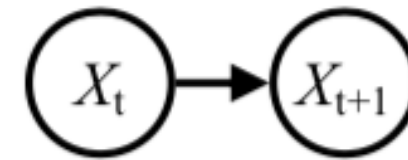


$$\begin{aligned} X_{t+1} &= A_t X_t + B_t u_t + \varepsilon_t & \varepsilon_t &\sim \mathcal{N}(0, Q_t) \\ Z_t &= C_t X_t + d_t + \delta_t & \delta_t &\sim \mathcal{N}(0, R_t) \end{aligned}$$

Dynamics (Action) Update

- Assume we have current belief for $X_{t|0:t}$:

$$p(x_t|z_{0:t}, u_{0:t})$$



Update the belief using action

- Then, after one time step passes:

Marginalize out x_t

$$p(x_{t+1}|z_{0:t}, u_{0:t}) = \int_{x_t} p(x_{t+1}, x_t|z_{0:t}, u_{0:t}) dx_t$$

Apply conditional independence

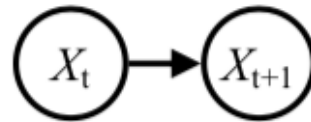
$$\begin{aligned} p(x_{t+1}, x_t|z_{0:t}, u_{0:t}) &= p(x_{t+1}|x_t, z_{0:t}, u_{0:t})p(x_t|z_{0:t}, u_{0:t}) \\ &= p(x_{t+1}|x_t, u_t) p(x_t|z_{0:t}, u_{0:t}) \end{aligned}$$

Product of two Gaussian distributions. We know that this is a Gaussian distribution.

Dynamics (Action) Update

- Assume we have current belief for $X_{t|0:t}$:

$$p(x_t|z_{0:t}, u_{0:t})$$



Update the belief
using action

- Then, after one time step passes:

$$p(x_{t+1}|z_{0:t}, u_{0:t}) = \int_{x_t} p(x_{t+1}, x_t|z_{0:t}, u_{0:t}) dx_t$$



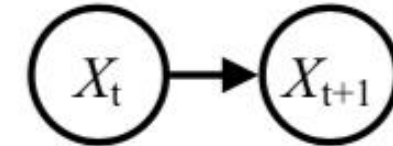
$$\begin{aligned} p(x_{t+1}, x_t|z_{0:t}, u_{0:t}) &= p(x_{t+1}|x_t, u_t) p(x_t|z_{0:t}, u_{0:t}) \\ &= \frac{1}{(2\pi)^{n/2} |\Sigma_{t|0:t}|^{1/2}} e^{-\frac{1}{2} (x_t - \mu_{t|0:t})^\top \Sigma_{t|0:t}^{-1} (x_t - \mu_{t|0:t})} \\ &\quad \frac{1}{(2\pi)^{n/2} |Q_t|^{1/2}} e^{-\frac{1}{2} (x_{t+1} - (A_t x_t + B_t u_t))^\top Q_t^{-1} (x_{t+1} - (A_t x_t + B_t u_t))} \end{aligned}$$

Product of two Gaussian distributions - a
Gaussian distribution.

Dynamics (Action) Update

- Assume we have

$$\begin{aligned}X_{t|0:t} &\sim \mathcal{N}(\mu_{t|0:t}, \Sigma_{t|0:t}) \\X_{t+1} &= A_t X_t + B_t u_t + \epsilon_t, \\ \epsilon_t &\sim \mathcal{N}(0, Q_t), \text{ and independent of } x_{0:t}, z_{0:t}, u_{0:t}, \epsilon_{0:t-1}\end{aligned}$$



- Then we have

$$\begin{aligned}(X_{t|0:t}, X_{t+1|0:t}) &\sim \mathcal{N}\left(\begin{bmatrix} \mu_{t|0:t} \\ \mu_{t+1|0:t} \end{bmatrix}, \begin{bmatrix} \Sigma_{t|0:t} & \Sigma_{t,t+1|0:t} \\ \Sigma_{t+1,t|0:t} & \Sigma_{t+1|0:t} \end{bmatrix}\right) \\ &= \mathcal{N}\left(\begin{bmatrix} \mu_{t|0:t} \\ A_t \mu_{t|0:t} + B_t u_t \end{bmatrix}, \begin{bmatrix} \Sigma_{t|0:t} & \Sigma_{t|0:t} A_t^\top \\ A_t \Sigma_{t|0:t} & A_t \Sigma_{t|0:t} A_t^\top + Q_t \end{bmatrix}\right)\end{aligned}$$

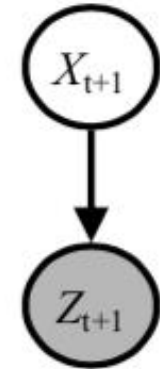
- Marginalizing the joint, we immediately get

$$X_{t+1|0:t} \sim \mathcal{N}(A_t \mu_{t|0:t} + B_t u_t, A_t \Sigma_{t|0:t} A_t^\top + Q_t)$$

A new Gaussian with the mean vector and the covariance matrix updated.

Measurement Update

Update the belief over the state by conditioning on the observation



- Assume we have:

$$\begin{aligned} X_{t+1|0:t} &\sim \mathcal{N}(\mu_{t+1|0:t}, \Sigma_{t+1|0:t}) \\ Z_{t+1} &\sim C_{t+1}X_{t+1} + d_{t+1} + \delta_{t+1} \\ \delta_{t+1} &\sim \mathcal{N}(0, R_t), \text{ and independent of } x_{0:t+1}, z_{0:t}, u_{0:t}, \epsilon_{0:t}, \end{aligned}$$

- Then:

$$(X_{t+1|0:t}, Z_{t+1|0:t}) \sim \mathcal{N}\left(\begin{bmatrix} \mu_{t+1|0:t} \\ C_{t+1}\mu_{t+1|0:t} + d \end{bmatrix}, \begin{bmatrix} \Sigma_{t+1|0:t} & \Sigma_{t+1|0:t}C_{t+1}^\top \\ C_{t+1}\Sigma_{t+1|0:t} & C_{t+1}\Sigma_{t+1|0:t}C_{t+1}^\top + R_{t+1} \end{bmatrix}\right)$$

- And, by conditioning on $Z_{t+1} = z_{t+1}$ (see lecture slides on Gaussians) we readily get:

$$\begin{aligned} X_{t+1|z_{0:t+1}, u_{0:t}} &= X_{t+1|0:t+1} \\ &\sim \mathcal{N}\left(\mu_{t+1|0:t} + \Sigma_{t+1|0:t}C_{t+1}^\top(C_{t+1}\Sigma_{t+1|0:t}C_{t+1}^\top + R_{t+1})^{-1}(z_{t+1} - (C_{t+1}\mu_{t+1|0:t} + d)), \right. \\ &\quad \left. \Sigma_{t+1|0:t} - \Sigma_{t+1|0:t}C_{t+1}^\top(C_{t+1}\Sigma_{t+1|0:t}C_{t+1}^\top + R_{t+1})^{-1}C_{t+1}\Sigma_{t+1|0:t}\right) \end{aligned}$$

Kalman Filter

Core Idea: Recursively update the mean and the covariance using the action model and the sensor model.

Initial belief is a Gaussian



■ At time 0: $X_0 \sim \mathcal{N}(\mu_{0|0}, \Sigma_{0|0})$

Belief always remains Gaussian



■ For $t = 1, 2, \dots$

Prediction

- What would be the next state belief under the process model?
- Updates the mean and inflates the covariance.



■ Dynamics update:

$$\mu_{t+1|0:t} = A_t \mu_{t|0:t} + B_t u_t$$

$$\Sigma_{t+1|0:t} = A_t \Sigma_{t|0:t} A_t^\top + Q_t$$

■ Measurement update:

$$\mu_{t+1|0:t+1} = \mu_{t+1|0:t} + \Sigma_{t+1|0:t} C_{t+1}^\top (C_{t+1} \Sigma_{t+1|0:t} C_{t+1}^\top + R_{t+1})^{-1} (z_{t+1} - (C_{t+1} \mu_{t+1|0:t} + d))$$

$$\Sigma_{t+1|0:t+1} = \Sigma_{t+1|0:t} - \Sigma_{t+1|0:t} C_{t+1}^\top (C_{t+1} \Sigma_{t+1|0:t} C_{t+1}^\top + R_{t+1})^{-1} C_{t+1} \Sigma_{t+1|0:t}$$

Correction

- Update the predicted belief with the observation.
- Updates the mean and deflates the covariance.



■ Often written as:

$$K_{t+1} = \Sigma_{t+1|0:t} C_{t+1}^\top (C_{t+1} \Sigma_{t+1|0:t} C_{t+1}^\top + R_{t+1})^{-1} \quad \text{(Kalman gain)}$$

$$\mu_{t+1|0:t+1} = \mu_{t+1|0:t} + K_{t+1} (z_{t+1} - (C_{t+1} \mu_{t+1|0:t} + d)) \quad \text{"innovation"}$$

$$\Sigma_{t+1|0:t+1} = (I - K_{t+1} C_{t+1}) \Sigma_{t+1|0:t}$$

Kalman Filter: Alternate Notation

Algorithm **Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

Previous belief action observation

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$$

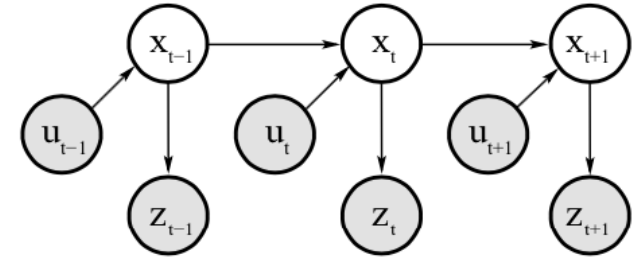
$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$$

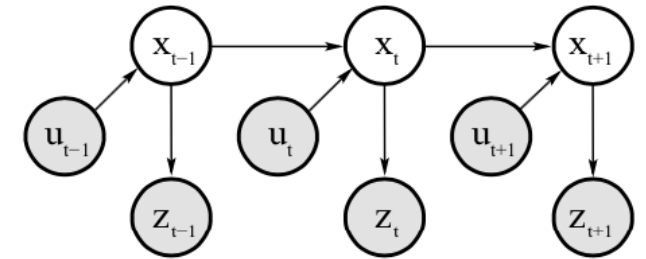
$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

return μ_t, Σ_t

Belief is gaussian



Kalman Filter



Algorithm Kalman_filter($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$$

$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

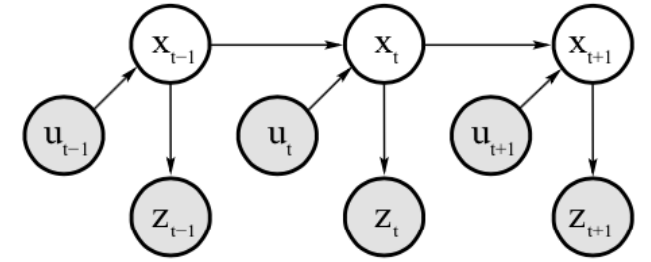
$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

return μ_t, Σ_t

Action

Kalman Filter



Algorithm Kalman_filter($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$$

$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \longleftarrow$$

Kalman gain:
Degree at which
observation factors into
belief

$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

return μ_t, Σ_t

Kalman Filter

Algorithm Kalman_filter($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$$

$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

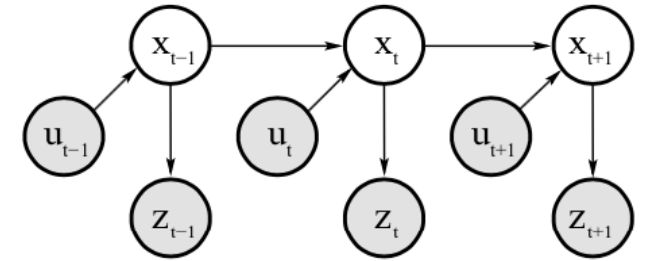
$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

return μ_t, Σ_t

Compute mean from
difference between
expected and observed
observations multiplied
by Kalman Gain

“innovation”



Kalman Filter: Constant Velocity Case

- $X = [x, y, v_x, v_y]$
- Constant velocity motion:

$$f(X, v) = [x + \Delta t \cdot v_x, y + \Delta t \cdot v_y, v_x, v_y] + v$$

$$v \sim N(0, Q)$$

$$Q = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & q^2 & 0 \\ 0 & 0 & 0 & q^2 \end{pmatrix}$$

- Only position is observed:

$$z = h(X, w) = [x, y] + w$$

$$w \sim N(0, R) \quad R = \begin{pmatrix} r^2 & 0 \\ 0 & r^2 \end{pmatrix}$$

Kalman Filter: Constant Velocity Case

$$f(X, v) = [x + \Delta t \cdot v_x, y + \Delta t \cdot v_y, v_x, v_y] + v$$

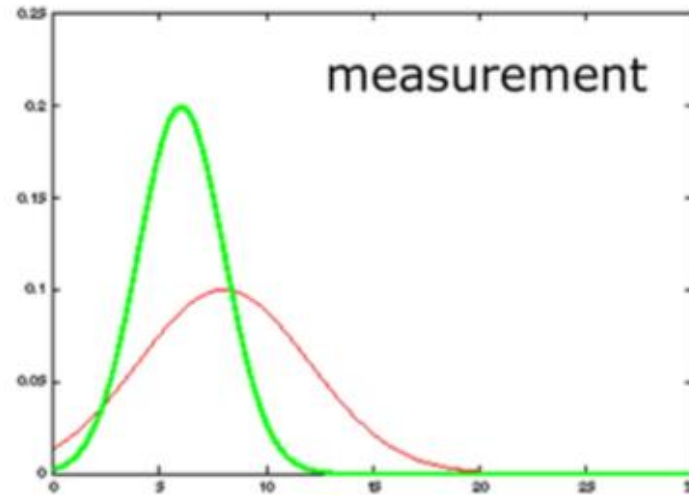
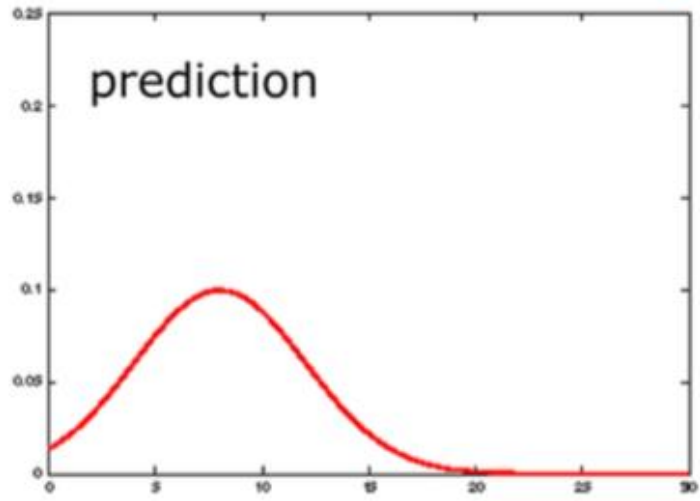
$$z = h(X, w) = [x, y] + w$$

$$\begin{pmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{A_t} \begin{pmatrix} x_k \\ y_k \\ v_{x,k-1} \\ v_{y,k-1} \end{pmatrix} + N(0, Q_k)$$

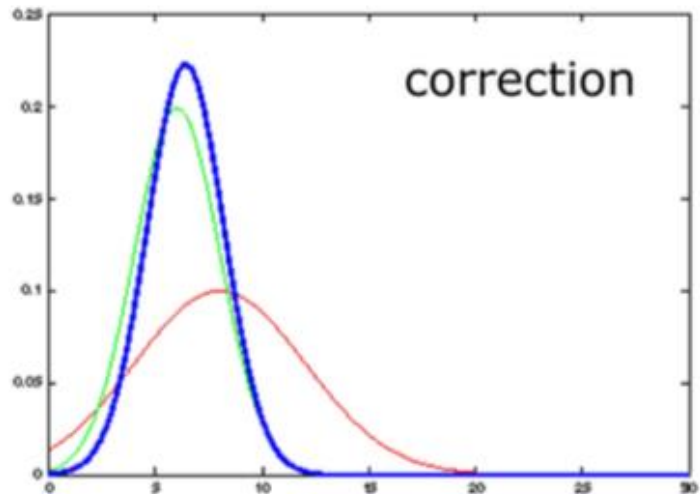
$$\begin{pmatrix} x_{obs} \\ y_{obs} \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}}_{C_t} \begin{pmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \end{pmatrix} + N(0, R_k)$$

If there were actions (e.g., changes to velocity) then the B matrix would be added in the motion model.

Example: 1D Gaussian Case



The corrected mean lies between the predicted and the mean of the measurement model. Weighted sum.



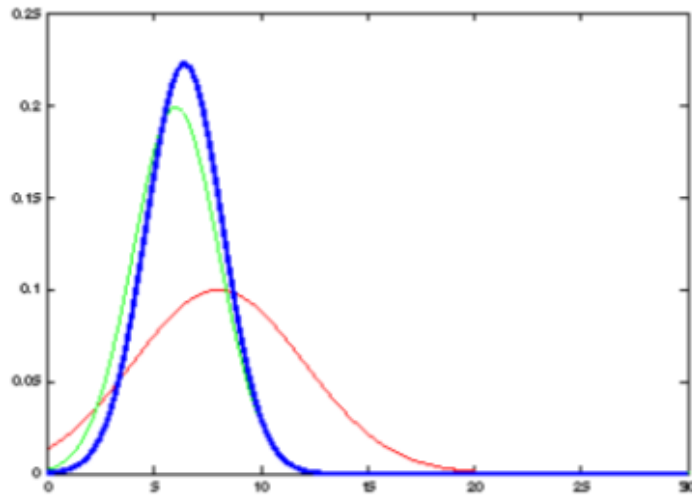
$$bel(x_t) = \begin{cases} \mu_t = \bar{\mu}_t + K_t(z_t - \bar{\mu}_t) \\ \sigma_t^2 = (1 - K_t)\bar{\sigma}_t^2 \end{cases}$$

$$\text{with } K_t = \frac{\bar{\sigma}_t^2}{\bar{\sigma}_t^2 + \bar{\sigma}_{obs,t}^2}$$

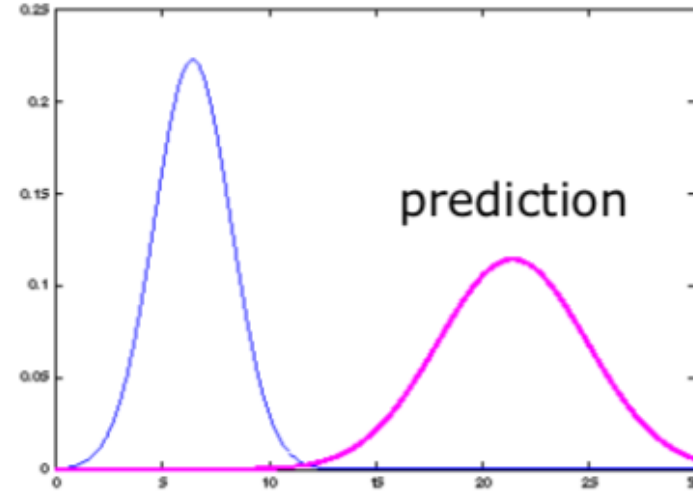
$$bel(x_t) = \begin{cases} \mu_t = \bar{\mu}_t + K_t(z_t - C_t\bar{\mu}_t) \\ \Sigma_t = (I - K_tC_t)\bar{\Sigma}_t \end{cases}$$

$$\text{with } K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + R_t)^{-1}$$

Example: 1D Gaussian Case



Belief after last measurement update.

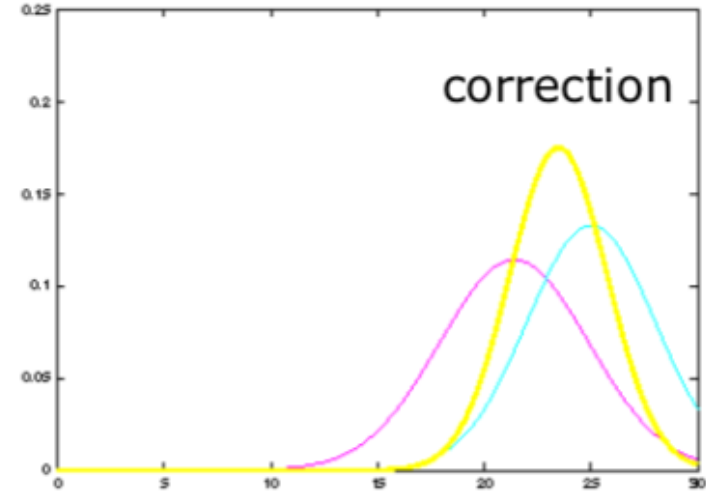
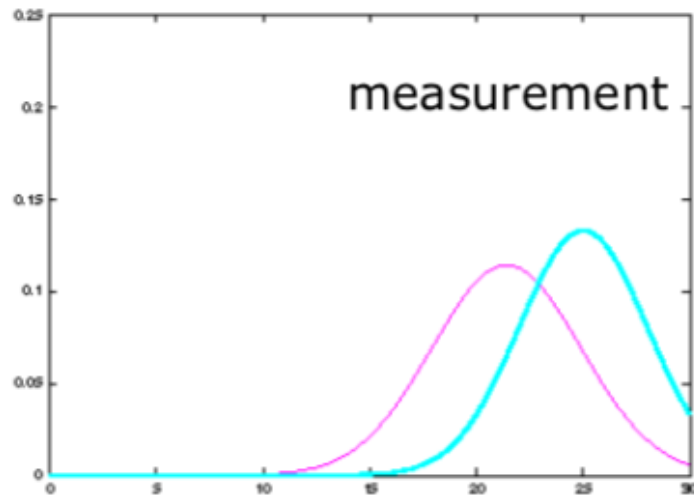
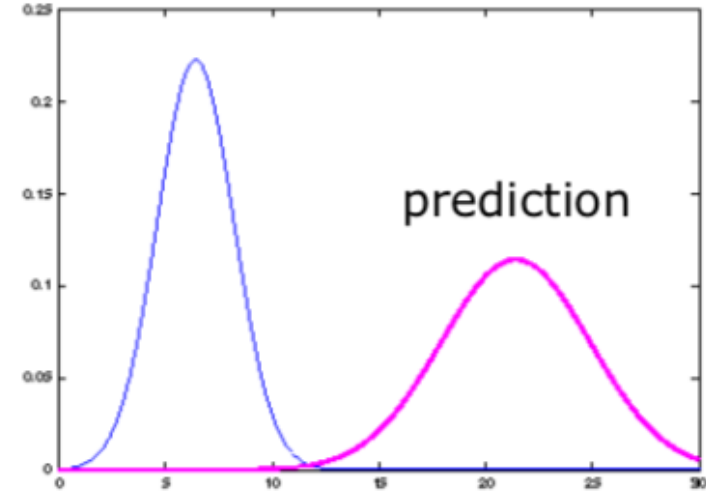
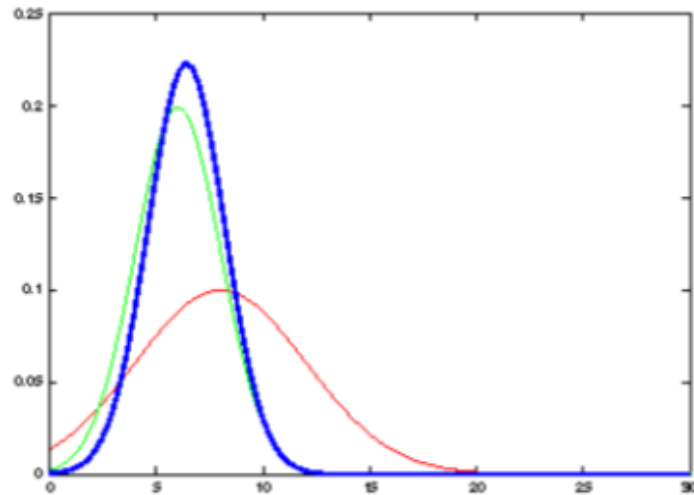


Magenta is the state after the prediction step is applied. The belief becomes less – localized.

$$\overline{bel}(x_t) = \begin{cases} \bar{\mu}_t = a_t \mu_{t-1} + b_t u_t \\ \bar{\sigma}_t^2 = a_t^2 \sigma_t^2 + \sigma_{act,t}^2 \end{cases}$$

$$\overline{bel}(x_t) = \begin{cases} \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + Q_t \end{cases}$$

Example: 1D Gaussian Case



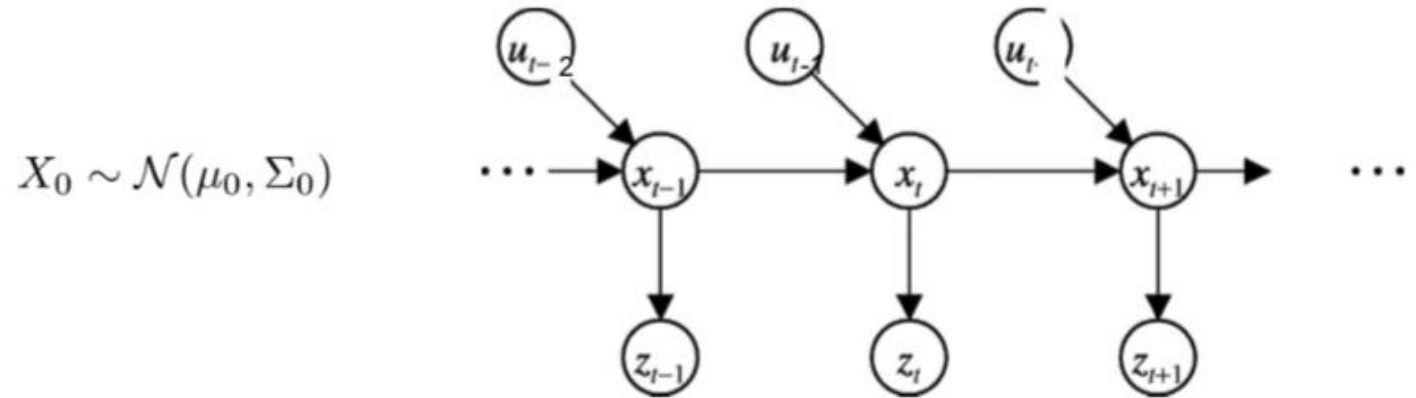
New measurement
and correction.

Kalman Filter: Other Takeaways

- **Optimal estimator**
 - Kalman filter is the optimal estimator for linear Gaussian case (i.e., we can't do better under the assumptions).
- **Efficient**
 - Polynomial in the measurement dimensionality k and the state dimensionality n : $O(k^{2.376} + n^2)$
- **Structure**
 - Asynchronicity: if no observations then propagate the motion model.
 - The measurement need not fully determine the latent state. Inherently, updating with partial observations.
 - Requires an initial prior mean and covariance. Predictor and corrector architecture.
- **Assumes and maintains a Gaussian Belief**
 - Unimodal and Gaussian.
 - Problem: in real life belief is often non-Gaussian and multi-modal.

Non-linearity: Extended Kalman Filter

- Kalman Filter (KF)
 - Assumed linear motion and observation models.
- Non-linearity
 - In several cases the sensor and the motion may be non-linear.
- Extended Kalman Filter
 - The EKF provides a way to handle non-linear motion and observation models.
 - “Extends” the use of the KF to non-linear problems.



$$\begin{aligned} X_{t+1} &= A_t X_t + B_t u_t + \varepsilon_t & \varepsilon_t &\sim \mathcal{N}(0, Q_t) \\ Z_t &= C_t X_t + d_t + \delta_t & \delta_t &\sim \mathcal{N}(0, R_t) \end{aligned}$$

Non-linear Models

- Non-linear setting

- The next state is a non-linear function of the current state and actions.
 - *Example: if the control input is a velocity then the velocity components have cosine/sine terms.*
- The observation is a non-linear function of the state.
 - *Example: observation is a distance to a landmark instead of (x,y) positions. Distance is a non-linear operation.*

$$X_{t+1} = f_t(X_t, u_t) + \varepsilon_t \quad \varepsilon_t \sim \mathcal{N}(0, Q_t)$$

$$Z_t = h_t(X_t) + \delta_t \quad \delta_t \sim \mathcal{N}(0, R_t)$$

- Linear setting

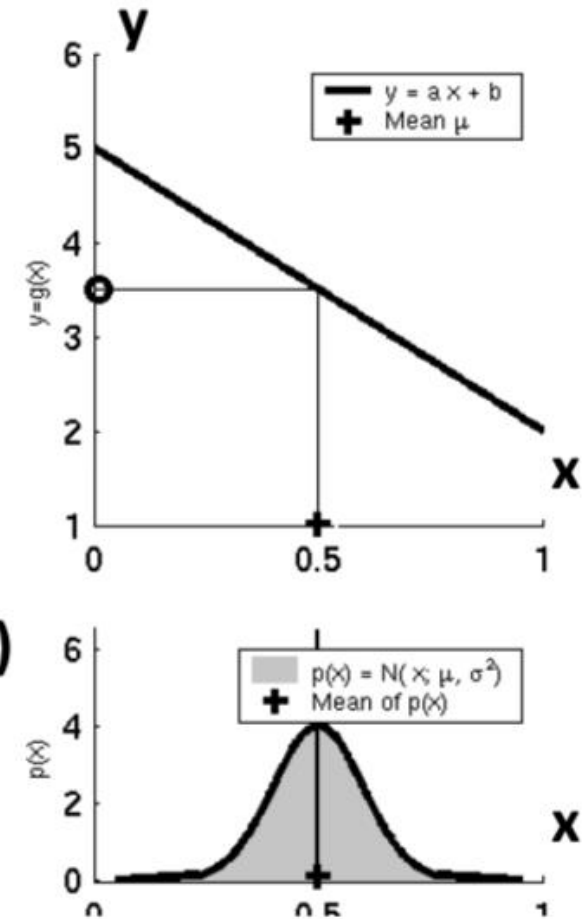
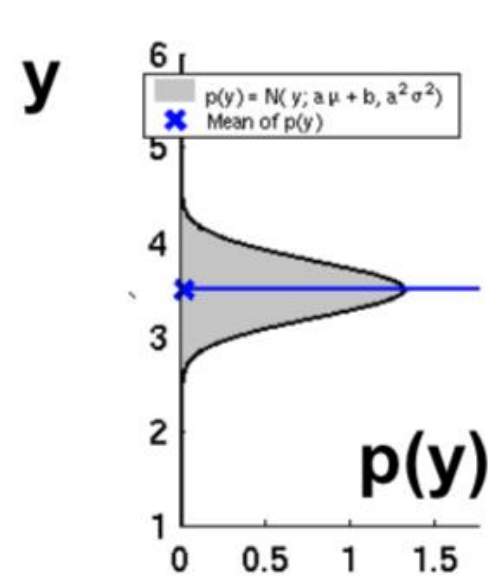
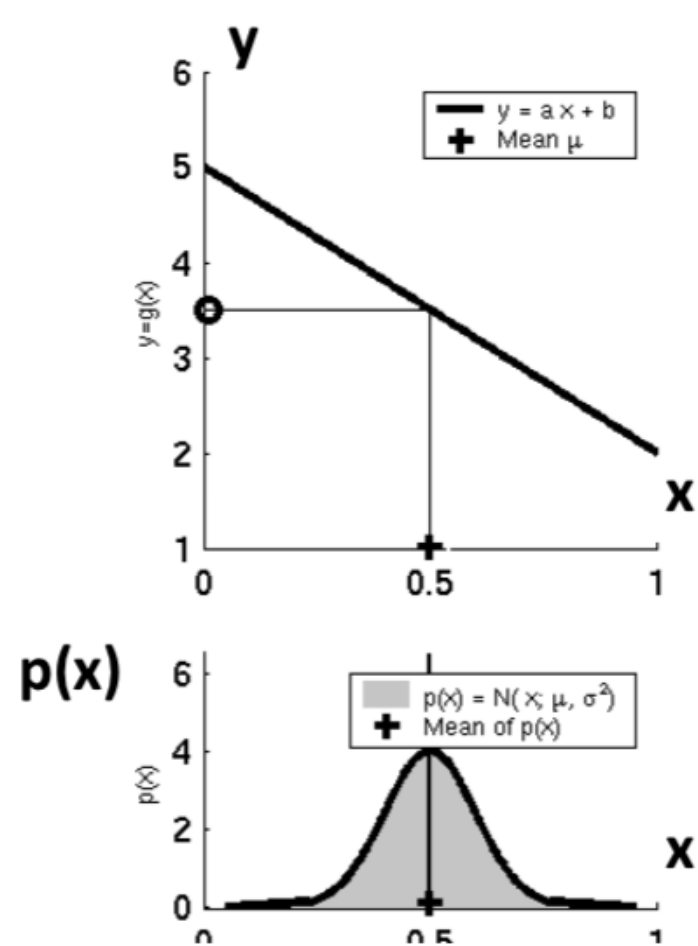
- As discussed for KF.

$$X_{t+1} = A_t X_t + B_t u_t + \varepsilon_t \quad \varepsilon_t \sim \mathcal{N}(0, Q_t)$$

$$Z_t = C_t X_t + d_t + \delta_t \quad \delta_t \sim \mathcal{N}(0, R_t)$$

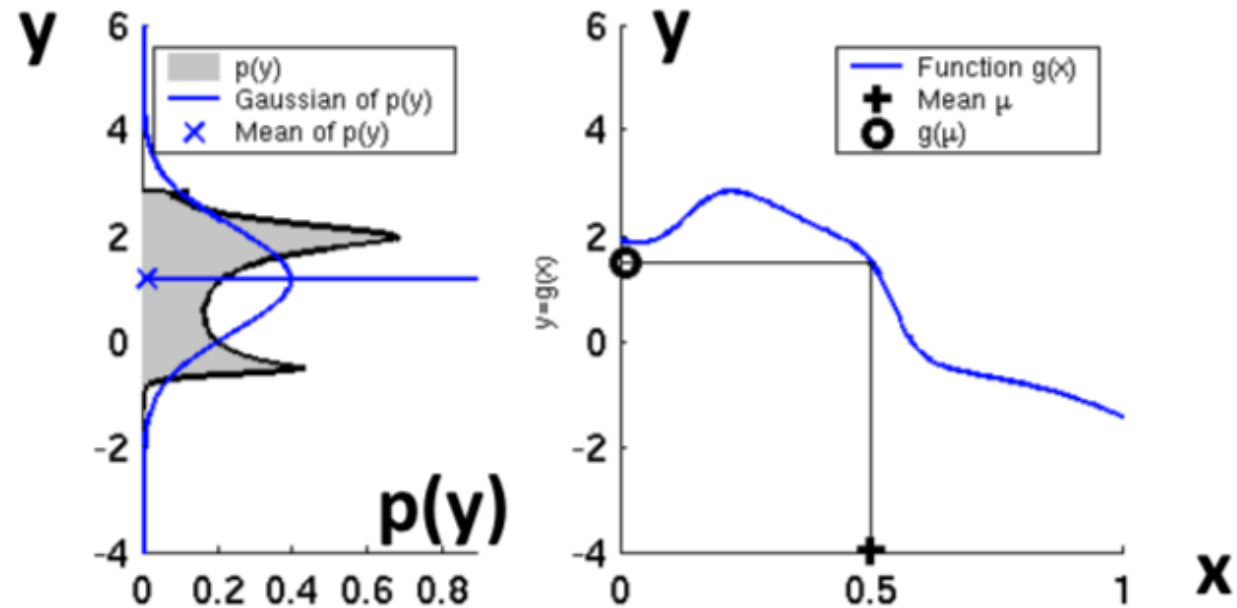
How do we update the belief over the state when there are non-linear dynamics and measurement functions are present?

Applying a linear function on a Gaussian Belief

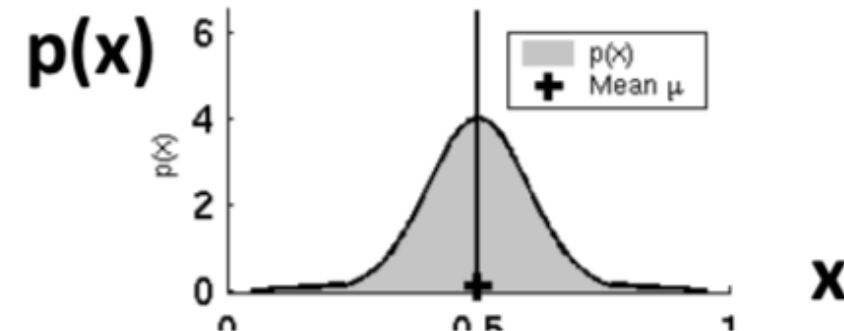


Applying a *non-linear* function on a Gaussian Belief

- A Gaussian random variable passed through a non-linear transformation.

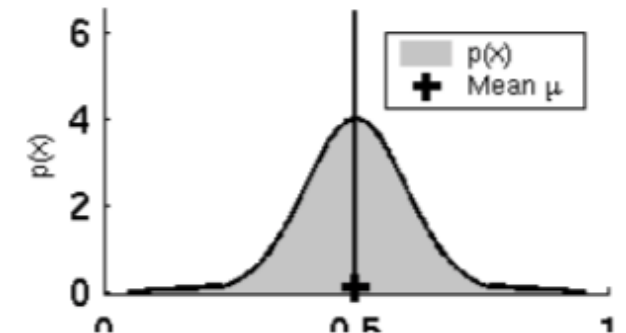
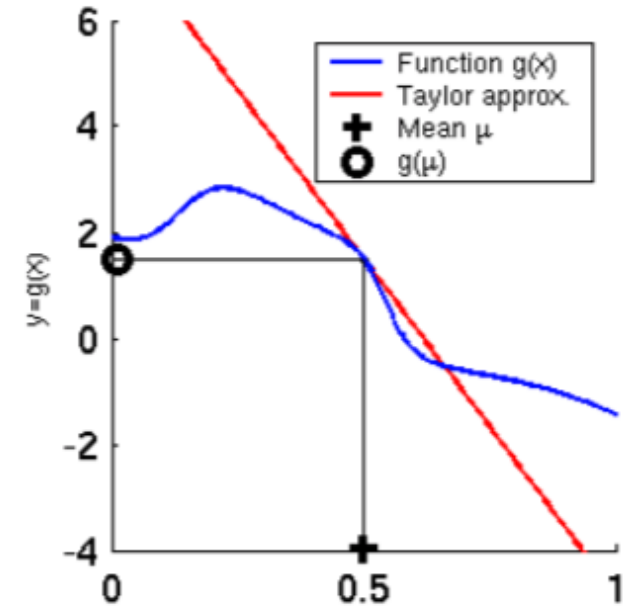
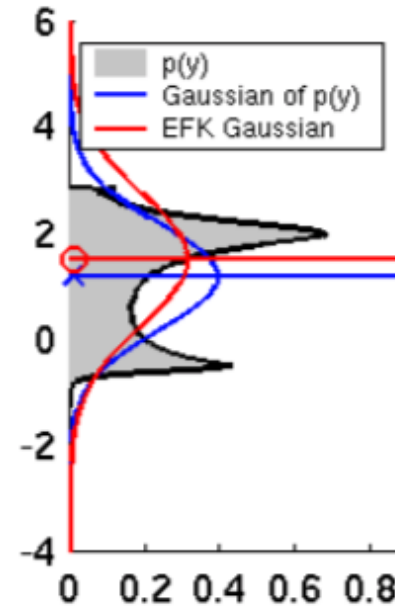


“Gaussian of $p(y)$ ” has mean and variance of y under $p(y)$



EKF Linearization

- Problem
 - With a non-linear transformation, the resulting belief is non-Gaussian.
- Solution
 - Can the non-linear function be **linearized** or (locally) approximated as a linear function?
 - Once linearized, the transformed belief can be approximated as a Gaussian.
- EKF Linearization
 - Instead of passing the Gaussian through a non-linear function, pass it through a locally linear approximation to the function.



EKF Linearization: First-Order Taylor Series Expansion

- **Dynamics model:** for x_t “close to” μ_t we have:

$$\begin{aligned} f_t(x_t, u_t) &\approx f_t(\mu_t, u_t) + \frac{\partial f_t(\mu_t, u_t)}{\partial x_t} (x_t - \mu_t) \\ &= f_t(\mu_t, u_t) + F_t(x_t - \mu_t) \end{aligned}$$

- **Measurement model:** for x_t “close to” μ_t we have:

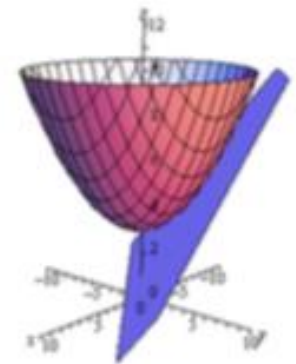
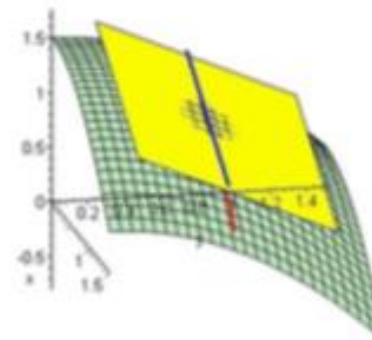
$$\begin{aligned} h_t(x_t) &\approx h_t(\mu_t) + \frac{\partial h_t(\mu_t)}{\partial x_t} (x_t - \mu_t) \\ &= h_t(\mu_t) + H_t(x_t - \mu_t) \end{aligned}$$

Note: linearization is around the current mean estimate of the belief over the state.

Jacobian Matrix

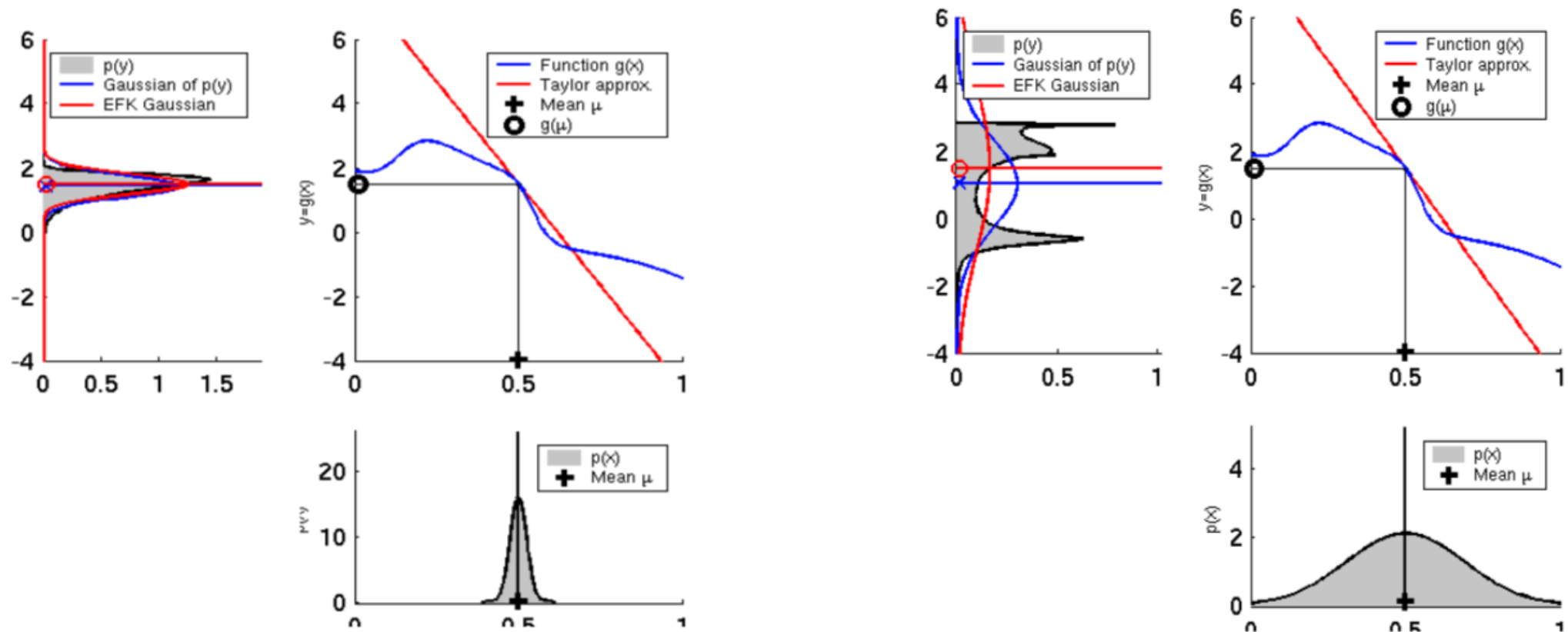
- Given a vector valued function $f(\mathbf{x})$ from dimension n to m .
- The Jacobian matrix $F_{\mathbf{x}}$ is of size $(n \times m)$.
- The orientation of the tangent plane to the vector-valued function at a given point
- Generalizes the gradient of a scalar valued function

$$f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix} \quad F_{\mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$



EKF Linearization

- Dependence of the approximation quality on the uncertainty.
- Cases: when $p(X)$ initial belief has low and high variance relative to the region in which the linearization is accurate.



EKF Algorithm

- At time 0: $X_0 \sim \mathcal{N}(\mu_{0|0}, \Sigma_{0|0})$
- For $t = 1, 2, \dots$

- Dynamics update:

$$f_t(x_t, u_t) \approx a_{0,t} + F_t(x_t - \mu_{t|0:t})$$

$$(a_{0,t}, F_t) = \text{linearize}(f_t, \mu_{t|0:t}, \Sigma_{t|0:t}, u_t)$$

$$\mu_{t+1|0:t} = a_{0,t}$$

$$\Sigma_{t+1|0:t} = F_t \Sigma_{t|0:t} F_t^\top + Q_t$$

- Measurement update:

$$h_{t+1}(x_{t+1}) \approx c_{0,t+1} + H_{t+1}(x_{t+1} - \mu_{t+1|0:t})$$

$$(c_{0,t+1}, H_{t+1}) = \text{linearize}(h_{t+1}, \mu_{t+1|0:t}, \Sigma_{t+1|0:t})$$

$$K_{t+1} = \Sigma_{t+1|0:t} H_{t+1}^\top (H_{t+1} \Sigma_{t+1|0:t} H_{t+1}^\top + R_{t+1})^{-1}$$

$$\mu_{t+1|0:t+1} = \mu_{t+1|0:t} + K_{t+1}(z_{t+1} - c_{0,t+1})$$

$$\Sigma_{t+1|0:t+1} = (I - K_{t+1} H_{t+1}) \Sigma_{t+1|0:t}$$

EKF Algorithm

Linearization of the motion and the observation models.

- Prediction:

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}} (x_{t-1} - \mu_{t-1})$$

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + G_t (x_{t-1} - \mu_{t-1})$$

- Correction:

$$h(x_t) \approx h(\bar{\mu}_t) + \frac{\partial h(\bar{\mu}_t)}{\partial x_t} (x_t - \bar{\mu}_t)$$

$$h(x_t) \approx h(\bar{\mu}_t) + H_t (x_t - \bar{\mu}_t)$$

Jacobian matrices

Once the motion and the observation models have been linearized, perform the similar updates as the Kalman Filter.

1. **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

2. Prediction:

3. $\bar{\mu}_t = g(u_t, \mu_{t-1})$ $\longleftarrow \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$

4. $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + Q_t$ $\longleftarrow \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + Q_t$

5. Correction:

6. $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + R_t)^{-1}$ $\longleftarrow K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + R_t)^{-1}$

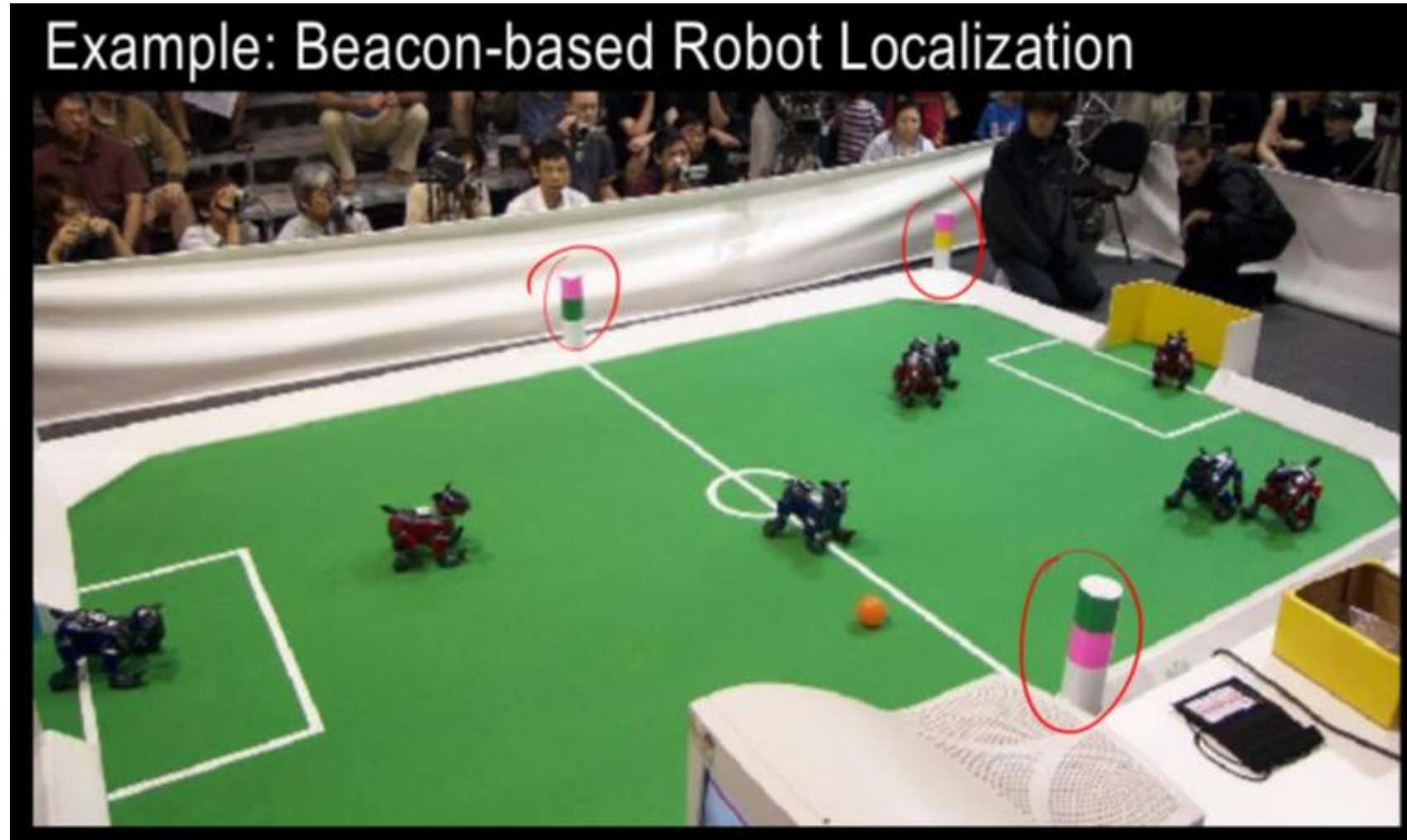
7. $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$ $\longleftarrow \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$

8. $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$ $\longleftarrow \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$

9. **Return** μ_t, Σ_t

$$H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t} \quad G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}$$

Application



Application

Example Motion Model

- State is $x_t = (x_t, y_t, \theta_t)$
- Command is rotation, translation, rotation

$$u_t = (\delta_{rot_1}, \delta_{trans}, \delta_{rot_2})$$

- Actual motion is $(\tilde{\delta}_{rot_1}, \tilde{\delta}_{trans}, \tilde{\delta}_{rot_2})$, a noisy version of the command

- Motion model g is:

$$x_{t+1} = x_t + \tilde{\delta}_{trans} \cos(\theta_t + \tilde{\delta}_{rot_1})$$

$$y_{t+1} = y_t + \tilde{\delta}_{trans} \sin(\theta_t + \tilde{\delta}_{rot_1})$$

$$\theta_{t+1} = \theta_t + \tilde{\delta}_{rot_1} + \tilde{\delta}_{rot_2}$$



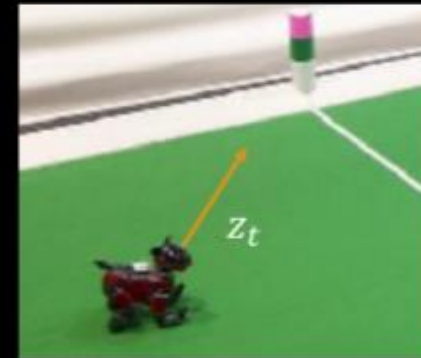
Application

Example sensor model

- The map is known
 - Beacons are at known positions
- Sensor reports noisy bearing $\tilde{\theta}$ and exact landmark ID L
 - Only one beacon is observed at one time

$$z_t = \begin{pmatrix} \tilde{\theta} \\ L \end{pmatrix} = \begin{pmatrix} \text{atan2}(y_{rob} - y_L, x_{rob} - x_L) \\ L \end{pmatrix}$$

Not linear!



discrete

EKF: Other Takeaways

- Non-optimal.
 - EKF is *approximate* and can diverge if the non-linearities are large.
 - Note that Kalman Filter was the optimal filter.
- Effectiveness
 - Handles Non-Gaussian sensor and motion models.
 - Note: still does not handle multi-modality (other methods such as histogram filters and particle filters that address multi-modality).
- Efficient
 - Polynomial in the measurement dimensionality k and the state dimensionality n : $O(k^{2.376} + n^2)$